

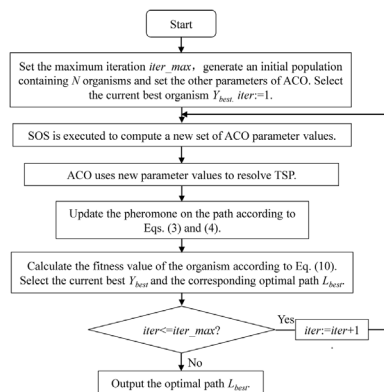
Ant colony optimization for traveling salesman problem based on parameters optimization

Yong Wang^{a,b,*}, Zunpu Han^a

^a School of New Energy, North China Electric Power University, Beijing 102206, China

^b College of Information Engineering, Tarim University, Alar, Xinjiang 843300, China

GRAPHICAL ABSTRACT



ARTICLE INFO

Article history:

Received 6 July 2020

Received in revised form 12 March 2021

Accepted 14 April 2021

Available online 5 May 2021

Keywords:

Traveling salesman problem

Symbiotic organisms search

Ant colony optimization

Parameter optimization

ABSTRACT

Traveling salesman problem (TSP) is one typical combinatorial optimization problem. Ant colony optimization (ACO) is useful for solving discrete optimization problems whereas the performance of ACO depends on the values of parameters. The hybrid symbiotic organisms search (SOS) and ACO algorithm (SOS-ACO) is proposed for TSP. After certain parameters of ACO are assigned, the remaining parameters can be adaptively optimized by SOS. Using the optimized parameters, ACO finds the optimal or near-optimal solution and the complexity for assigning ACO parameters is greatly reduced. In addition, one simple local optimization strategy is incorporated into SOS-ACO for improving the convergence rate and solution quality. SOS-ACO is tested with different TSP instances in TSPLIB. The best solutions are within 2.33% of the known optimal solution. Compared with some of the previous algorithms, SOS-ACO finds the better solutions under the same preconditions. Finally, the performance of SOS-ACO is analyzed according to the changes of some ACO parameters. The experimental results illustrate that SOS-ACO has good adaptive ability to various values of these parameters for finding the competitive solutions.

© 2021 Elsevier B.V. All rights reserved.

* Corresponding author at: School of New Energy, North China Electric Power University, Beijing 102206, China.

E-mail address: wangyong100@126.com (Y. Wang).

1. Introduction

Traveling salesman problem (TSP) is a classical combinatorial optimization problem [1]. Given a set of cities and the distance between each pair of cities, a salesman starts from an initial city and visits each city once, and then returns to the initial city. The salesman's route forms a Hamiltonian Cycle (HC). The objective

of TSP is to find the Optimal Hamiltonian Cycle (OHC) with the minimum distance [2]. Many engineering problems, such as vehicle routing, electronic circuit design, logistics distribution, and network optimization, etc., can be converted into TSP [3–5].

In the computational complexity, TSP is an NP-complete problem [6]. The running time of the exact algorithms for TSP will increase exponentially in proportion to the city number. Researchers have proposed various methods and algorithms for TSP. In early stage, the exact methods are focused on by researchers, such as the integer linear programming, dynamic programming and graph algorithms [7,8]. The exact algorithms are able to find the optimal solution. However, they are apt to trap into the search space of combination explosion. Therefore, many researchers turn to the heuristic algorithms or approximate algorithms for TSP. Different from the exact methods, the heuristic algorithms can find a satisfactory solution or near-optimal solution within an acceptable computation time. The well-known heuristic algorithms include ACO, genetic algorithm (GA), particle swarm optimization (PSO), simulated annealing (SA) and etc. The literature review is given as follows.

Grefenstette et al. [9] proposed GA for TSP in which several transform operators are constructed for improving the solutions and the representation of HCs is discussed. Zhong et al. [10] introduced a discrete comprehensive learning particle swarm optimization algorithm for TSP where the good solutions are accepted based on Metropolis acceptance criterion. Geng et al. [11] combined SA and greedy search strategy for TSP. Three different probabilities are adopted in order to improve the solutions. In addition, the greedy search is used to speed up the convergence. Inspired by the behavior of ants, Dorigo and Gambardella [12] proposed ACO for TSP. In the computation process, the ants are induced by the pheromones on the paths for finding the satisfactory solutions. Elloumi et al. [13] proposed the hybrid algorithm combining ACO and PSO for TSP where PSO uses the pheromone deposit in ACO as a weight of particles to improve the performance. The experimental results illustrate the effectiveness of the hybrid method. Besides the above algorithms, the heuristic algorithms emerged in recent years were also applied to TSP. Khan and Maiti [14] improved the artificial bee colony (ABC) algorithm by modifying multiple update rules and adding the 3-Opt rule for TSP. Compared with the existing algorithms, the improved ABC can achieve better solutions. Akhand et al. [15] gave a discrete spider monkey optimization for TSP in which each spider monkey represents a solution of TSP and the spider monkey is updated according to the local leader and the global leader solutions at each iteration. Osaba et al. [16] designed a discrete water cycle algorithm for resolving the symmetric and asymmetric TSP. The exploration and exploitation capabilities of the algorithm are enhanced through adaptive modification of the movement function. A-Star (A^*) algorithm is a well-known heuristic algorithm which has been widely applied to TSP and other problems [17]. In [18], Mahafzah presented a parallel multi-threaded IDA* (PMIDA*) algorithm for TSP. Compared with some other algorithms, PMIDA* algorithm shows good performance for finding the better solutions. In addition, there are some other heuristic algorithms, such as whale optimization algorithm [19, 20], gray wolf optimization [21] and chemical reaction optimization [22], have been applied to TSP and other optimization problems.

Local search strategies are widely used to improve the performance of heuristic algorithms for TSP [23]. Croes [24] proposed k -Opt algorithm (k -Opt) in which k edges/connections in the previous solution are replaced with another k edges/connections for generating a better solution. 2-Opt is a particular case of k -Opt where two edges in the solution are replaced with another two new edges and a new solution is computed [25]. Dahan et al. [26]

incorporated 3-Opt into a dynamic flying ant colony optimization for TSP, and 3-Opt is used to improve the solution. Wang [27] improved GA with two local optimization strategies for TSP. The first local strategy is the four vertices and three lines inequality which is used to adjust the local paths containing four vertices for generating the shorter paths. The second local strategy is used to reverse the local paths containing more than two vertices/points. The two local optimization strategies collaborate with each other to produce the better and better solutions in the optimization process. In [28], a memetic ACO algorithm containing a local search operator is proposed for dynamic traveling salesman problem (DTSP). The best solution found by ACO is improved by the local search operator through removing and inserting cities. The experimental results illustrated the efficiency of the memetic ACO for DTSP.

The performance of heuristic algorithms depends on their global exploration ability and local exploitation ability. The exploration ability is helpful to find the global optimal solution whereas a large range of exploration will lower the convergence rate. On the other hand, the exploitation ability is useful to improve the convergence rate whereas it will cause the algorithm to fall into the local minima [29]. Therefore, in order to improve the heuristic algorithm, it is necessary to balance the functions of the exploration and exploitation. Besides the improvement of the algorithm mechanism, the rational assignment for algorithm parameters is another way to improve their performance [30]. In ACO, the behavior of ants is mainly determined by heuristic information and pheromone on the paths. The parameter values of the basic ACO are usually fixed in the optimization process. If the parameter values are not harmonious, the ants will dependent on either the heuristic information or the pheromone and the function of the other will be lowered. In this case, ACO generally traps into local minima. Moreover, the accurate parameter values are often different for different problems, even for different instances of the same type. Since the coupling relationships between the parameters are not known, there are no theories for assigning the ACO parameters in current stage. Given a problem, it usually requires many experiments for obtaining the useful parameter values [31].

In recent years, the parameter settings of ACO have been taken into account. Gan et al. [32] adopted the orthogonal experiment method to compute the parameters of ACO in order to improve the performance. Peker et al. [31] introduced the Taguchi method for determining the values of parameters for ACO. Experiments showed that Taguchi method can find the better parameter configuration in a fewer experiments. PSO is used to optimize the parameters of ACO by [33] and the 3-Opt was used to improve the solutions. In the similar way, Botee and Bonabeau [34] optimized the parameters of ACO with GA. In addition, some researchers applied machine learning algorithms to compute the ACO parameters. For example, Erol et al. [35] used artificial neural networks to optimize the two importance factors of the pheromone and heuristic information of ACO. First, ACO runs with different values of the two importance factors. Then, the artificial neural network predicts the possible optimal parameter values according to the solutions.

The above algorithms used to optimize the ACO parameters, such as PSO and GA, also have parameters. These parameters are often given by designers and the assignments will affect the performance of the hybrid algorithms. Different from most heuristic algorithms, SOS has an advantage of without parameters owing to Cheng and Prayogo [36]. Thus, the performance of SOS will not be influenced by parameter settings. SOS has been used to resolve various continuous and combinatorial optimization problems. Ezugwu and Adewumi [37] proposed a discrete symbiotic organism search (DSOS) for TSP. The mutualism,

commensalism, and parasitism phrases in SOS are repeatedly used to generate the better and better solutions. In addition, three mutation-based local search operators are used to improve the exploration and exploitation capability of DSOS. Wang and Wu [38] improved the DSOS algorithm using excellent coefficients and self-escape strategy, and proposed an ECSDSOS algorithm for TSP. The excellent coefficients improve the local optimization capability of DSOS, and the self-escape strategy avoids DSOS from falling into the local optimum. Panda and Pani [39] proposed a multi-objective symbiotic organisms search algorithm with adaptive penalty function to resolve multi-objective constrained optimization problems. The performance of the proposed algorithm was verified by twelve benchmark unconstrained and six constrained multi-objective optimization problems.

SOS does not contain any parameters. If SOS is used to optimize ACO parameters, the complexity of ACO parameter settings can be reduced. In view of the merits of SOS, a hybrid SOS-ACO algorithm is designed for TSP. In SOS-ACO, SOS is used to optimize the key parameters of ACO and ACO searches the optimal or near-optimal solutions for TSP using the optimized parameters. In addition, a common local optimization strategy is incorporated into the hybrid algorithm for increasing the convergence rate and the solution quality. The experimental results illustrated that SOS-ACO can search the optimal solution or approximation that is quite close to the optimal solution. Compared with the basic ACO and the ACO with the local optimization strategy (ACO-LO), SOS-ACO finds better solutions and the convergence rate is improved. Finally, the performance of the hybrid algorithm is analyzed according to the change of certain parameters. The experimental results show that SOS-ACO finds the satisfactory solutions under various values of these parameters. It says SOS-ACO has good adaptive capacity for resolving TSP.

The remainder of this paper is organized as follows. Section 2 briefly introduces the basic ACO. Section 3 describes the three stages of SOS. Section 4 introduces the local optimization strategy for improving the solutions. Section 5 gives the hybrid SOS-ACO algorithm. In Section 6, experiments are executed to validate the hybrid SOS-ACO. In the last Section, conclusions are drawn and the possible future research directions are proposed.

2. The basic ACO for TSP

In ACO, an ant k goes from the current city i to the next city j according to the state transition probability between i and j . As all cities are visited, an HC is found by this ant. The state transition probability depends on the pheromone on the path and heuristic information between i and j . It is computed as Eq. (1).

$$p_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}]^\beta}{\sum_{u \in allowed_k} [\tau_{iu}(t)]^\alpha [\eta_{iu}]^\beta} & j \in allowed_k \\ 0 & \text{Otherwise} \end{cases} \quad (1)$$

The variables in Eq. (1) are denoted as follows. $\tau_{ij}(t)$ is the pheromone left by the ants between cities i and j at the t th iteration. η_{ij} is the heuristic information between i and j , and it is computed as Eq. (2). α and β represent the importance factors of the pheromone and heuristic information, respectively, and they denote the relative importance of the pheromone and heuristic information for choosing the path between i and j . $allowed_k$ represents the candidate city set.

$$\eta_{ij} = \frac{1}{d_{ij}} \quad (2)$$

d_{ij} is the distance between cities i and j .

When all ants find their own HC, the pheromone on the paths will be updated. Each ant enhances the pheromone on the path

that it passes according to the quality of the obtained solution. Meanwhile, a part of pheromones on the paths will evaporate. The pheromone is updated as Eqs. (3) and (4).

$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k(t) \quad (3)$$

$$\Delta\tau_{ij}^k(t) = \begin{cases} \frac{Q}{L_k}, & \text{The } k\text{th ant passes the path between } i \text{ and } j \\ 0, & \text{Otherwise} \end{cases} \quad (4)$$

ρ is the pheromone evaporation coefficient. m is the number of ants. $\Delta\tau_{ij}^k(t)$ is the accumulated pheromone on the path between cities i and j and it is calculated by Eq. (4). In Eq. (4), L_k is the HC length found by the k th ant, Q is the pheromone enhancement coefficient which adjusts the increment of pheromone on the paths. The basic ACO is designed according to the above equations and the optimal or near-optimal path is found through iterative calculation.

3. The SOS algorithm for optimizing ACO parameters

In SOS, three phases mutualism, commensalism and parasitism are used to simulate the symbiotic relationships among organisms in nature. Different from most of the other intelligent algorithms, SOS does not contain any parameters. Since it is easy to implement and avoids resetting algorithm parameters, SOS is used to optimize the parameters of ACO.

During the iteration, SOS randomly generates a group of organisms as the initial population in an ecosystem. Each organism represents a solution of the problem. It has a fitness value which represents its adaptation to the ecosystem. The ancestor organisms evolve to the successor organisms through mutualism, commensalism and parasitism. In the evolution process, the organisms with a higher fitness value will be maintained to the next generation. The evolution is repeated until the termination conditions are met. At this time, the organism with the highest fitness value is output.

3.1. The mutualism phase

The mutualism phase imitates the mutualistic relationship between organisms in an ecosystem, such as the mutualism between rhinos and rhino birds, bees and flowers, etc. Given an organism X_i , another different organism $X_j \neq X_i$ is chosen from the population. A mutualism operation is performed for X_i and X_j in order to enhance their survival ability in the ecosystem. The new candidates $X_{i\text{new}}$ and $X_{j\text{new}}$ are computed according to Eqs. (5) and (6).

$$X_{i\text{new}} = X_i + \text{rand}(0, 1) \times (X_{\text{best}} - \text{Mutual_Vector} \times BF_1) \quad (5)$$

$$X_{j\text{new}} = X_j + \text{rand}(0, 1) \times (X_{\text{best}} - \text{Mutual_Vector} \times BF_2) \quad (6)$$

$$\text{Mutual_Vector} = \frac{X_i + X_j}{2} \quad (7)$$

$\text{rand}(0,1)$ is the random number generator which produces a random number in $(0, 1)$, X_{best} represents the organism with the highest fitness value at current iteration, $BF_1, BF_2 \in \{1, 2\}$ are the mutualism benefit factors between X_i and X_j . It mentions that BF_1 and BF_2 are two random numbers taken from either 1 or 2. Mutual_Vector is the common benefit determined by X_i and X_j . For continuous variables X_i and X_j , Mutual_Vector is generally their average value. $X_{i\text{new}}$ and $X_{j\text{new}}$ are accepted only if their fitness values are higher than those of the X_i and X_j , respectively. In the mutualism phase, the new solutions are computed according to the current optimal solution and the average of two organisms (Mutual_Vector). As both organisms are far from each other in the search space, new organisms will be explored from the regions between the two organisms.

3.2. The commensalism phase

Different from the mutualism phase, only one of the two organisms benefits from the commensalism phase whereas the other stays unchanged. Given an organism X_i , another organism X_j is selected at random from the population. X_i will be converted into a new organism under the help of X_j . The new organism is computed as Eq. (8).

$$X_{\text{new}} = X_i + \text{rand}(-1, 1) \times (X_{\text{best}} - X_j) \quad (8)$$

$\text{rand}(-1, 1)$ function produces a random number in $(-1, 1)$. The term $(X_{\text{best}} - X_j)$ represents the benefit to increase the survival ability of X_i . X_{new} will be accepted only if the fitness value is higher than that of the ancestor X_i . In the commensalism phase, the current best solution X_{best} is taken as the reference organism for updating X_i . It aims to compute a promising organism near X_{best} .

3.3. The parasitism phase

In this stage, one organism benefits from the parasitism operation which may do harm to the other. First, an organism X_i is selected and copied as an artificial parasite called *Parasite_Vector*. Then, *Parasite_Vector* is modified in some dimension computed with a random number function. At last, an organism X_j is selected as a host for comparison. If the *Parasite_Vector* has a better fitness value, it will replace X_j in the population and X_j will be deleted. Otherwise, X_j is maintained and *Parasite_Vector* will be neglected.

4. The local optimization strategy

In order to improve the solution quality, local search strategies are often combined with the heuristic algorithm for TSP. The quality of initial solutions plays an important role for local search strategies to find the satisfactory solutions quickly [28]. In general, a heuristic algorithm is used to compute the good initial solutions, and then some local search strategy is applied to improve the solutions. Literature [40] implements the reversal operator that the nearer cities will be connected with a higher probability. This local optimization strategy [40] is used to improve the solution quality searched by SOS-ACO and accelerate the convergence rate.

Given a TSP instance of size n , an HC is denoted as $\text{HC} = (v_1, v_2, \dots, v_n)$, where v_i is one of the n cities and i represents the position of city v_i in HC. There are two steps for improving an HC using the local optimization strategy.

Step 1: For each city v_k in an HC, find the city $v_{k'}$ which is nearest to v_k in the following cities. As shown in Eq. (9), $\text{dis}(v_i, v_j)$ is the distance between v_i and v_j .

$$\begin{cases} \text{dis}(v_k, v_{k'}) = \min \{\text{dis}(v_k, v_i)\} \\ i = k + 1, k + 2, \dots, n \end{cases} \quad (9)$$

Step 2: Reverse the path between v_k and $v_{k'}$ so that v_k is adjacent to $v_{k'}$. If the adjusted path is shorter than the original path, the adjusted path is preserved. Otherwise, the original path is maintained.

For example, an $\text{HC} = (6, 5, 7, 4, 1, 3, 2, 8, 9)$, when $k = 2$ and $k' = 6$, the new path is computed as $\text{HC}' = (6, 5, 3, 1, 4, 7, 2, 8, 9)$. Then, the shorter path will be preserved.

This local optimization strategy is a combination of nearest neighbor and path reversion. Using this local optimization strategy, the nearest cities will be adjacent to each other with a greater probability for obtaining a shorter path. The exploitation capability of SOS-ACO will be enhanced. In addition, it is apt to implement and has little impact on the time complexity of SOS-ACO. Therefore, this local optimization strategy is combined with SOS-ACO to improve the performance.

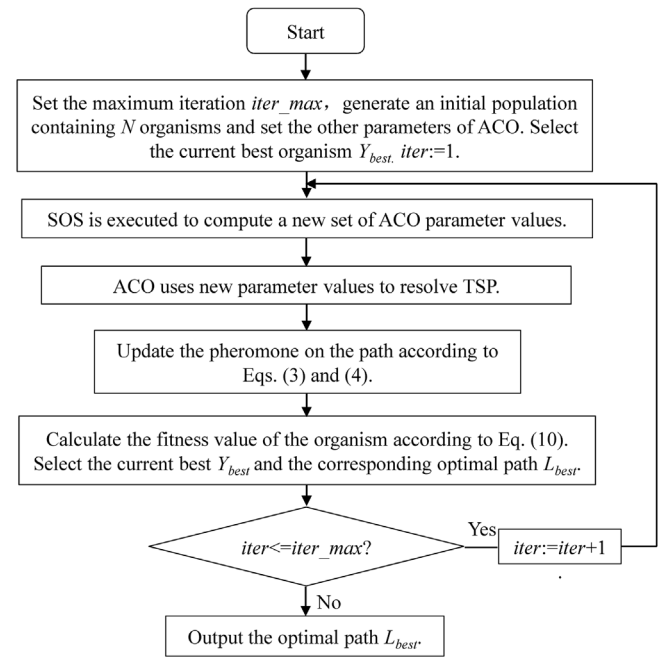


Fig. 1. The flowchart of the hybrid SOS-ACO algorithm.

5. The hybrid SOS-ACO

Before ACO is used to solve TSP, one must set the parameter values. Due to the coupling relationships among the parameters, it is time-consuming for finding a set of accurate parameter values for a given problem. However, the performance of ACO heavily depends on the parameter values. In order to reduce the complexity of parameter assignment, SOS is used to optimize the two key parameters α and β which have a big impact on the behavior of ants.

The organism $Y = [\alpha, \beta]$ in SOS is composed of α and β . The fitness value $\text{Fitness}(Y)$ of Y is calculated as Eq. (10).

$$\text{Fitness}(Y) = \frac{1}{d} \quad (10)$$

The definition of d is given as follows. As the number of ants is m , each ant uses α and β values in Y to find their own HC. After m HCs are obtained, the local optimization strategy is applied to the shortest HC for finding another best HC. d is the length of the best HC.

The organisms in SOS are updated through the three stages of mutualism, commensalism and parasitism. In the computation process, only the organisms with the higher fitness values are preserved. The flowchart of the hybrid SOS-ACO is shown in Fig. 1. As the terminal conditions are met, the procedure will be ended and the shortest HC will be output.

In the basic ACO, the parameter values are fixed. If the parameter values are not rational for an instance, ACO is apt to fall into local minima. Using the SOS-ACO, SOS will adjust the two key parameters α and β according to the HCs searched by the ants. Even if the pheromone on some paths reaches a high level, it is still possible for SOS to refine α and β values for balancing the guidance of pheromones and heuristic information to ants and preventing ants from repeating the previous paths. In this case, ant colony will keep exploring new HCs during the iteration and the local minima will be skipped out.

Table 1

Basic information of the TSP instances for resolving.

TSP	number of cities (n)	number of edges ($n(n-1)/2$)	BKS
Eil51	51	1275	426
Eil76	76	2850	538
KroA100	100	4950	21282
Ch150	150	11175	6528
KroA200	200	19900	29368
Pr264	264	34716	49135
Lin318	318	50403	42029
Rd400	400	79800	15281
Pr439	439	96141	107217
Rat575	575	165025	6773

Table 2

The assignments of parameters.

Parameter	α	β	$iter_max$	N	m	ρ	Q
Value	[1, 5]	[1, 5]	100/200/300	10	10	0.1	1000

6. Experiments and analysis

The hybrid SOS-ACO will be testified with different scales of TSP instances in TSPLIB [41]. For each TSP instance, SOS-ACO is run ten times in order to verify the stability of the algorithm. The experimental results are shown in Section 6.2. In Section 6.3, the experimental results of SOS-ACO are compared with those of some other algorithms. In Section 6.4, the performance of SOS-ACO is analyzed according to the changes of some parameters in ACO.

6.1. Experimental environment

In order to illustrate the performance of SOS-ACO for different scales of TSP, TSP instances of scales from 51 to 575 were selected from TSPLIB for resolving. The basic information of these TSP instances is shown in Table 1. BKS represents the length of the best-known solution of each TSP instance.

SOS-ACO is coded in C programming language and run on the computer with Intel(R) Core(TM) i5-5200U, CPU 2.20 GHz with 8 GB RAM. The value ranges of the parameters α and β and the other parameters are given in Table 2. For different scales of TSP, the parameters of SOS-ACO are slightly adjusted for reducing the computation time. When the scale n of TSP is smaller than 200, the maximum iteration $iter_max = 200$. As $200 \leq n < 400$ and $n \geq 400$, $iter_max$ is equal to 300 and 400, respectively. The population size N of SOS is 10, the number m of ants is 10, the pheromone evaporation coefficient ρ is 0.1, and the pheromone enhancement coefficient Q is 1000.

6.2. Experimental results of SOS-ACO

The experimental results are shown in Table 3. It mentions that all the decimals of BKS and Best lengths are neglected for comparisons. Best and Worst denote the best and worst HC among ten solutions for each TSP instance, respectively. Average is the average value of the ten solutions and SD denotes the standard deviation computed based on the ten solutions. Error(%) is the relative error of Average with respect to BKS. Error(%) is calculated as Eq. (11). The performance of SOS-ACO can be evaluated by Error(%). PE(%) is the relative error of Average with respect to Best. PE(%) is calculated as Eq. (12). The smaller the PE(%) is, the closer between Average and Best will be. The robustness of SOS-ACO can be evaluated by PE(%).

$$Error(\%) = \frac{Average - BKS}{BKS} \times 100\% \quad (11)$$

Table 3

Experimental results of SOS-ACO.

TSP	BKS	Best	Average	Worst	SD	Error(%)	PE(%)
Eil51	426	426	428.1	430	1.14	0.49	0.49
Eil76	538	538	541.7	545	2.49	0.69	0.69
KroA100	21282	21282	21290.1	21319	12.56	0.04	0.04
Ch150	6528	6558	6571.2	6586	10.52	0.66	0.20
KroA200	29368	29413	29520.2	29593	69.31	0.52	0.36
Pr264	49135	49135	49250.7	49432	107.6	0.24	0.24
Lin318	42029	42473	42762.7	42940	169.96	1.75	0.68
Rd400	15281	15429	15533	15608	60.66	1.65	0.67
Pr439	107217	107978	108873.8	109224	350.89	1.55	0.83
Rat575	6773	6887	6930.9	6964	30.08	2.33	0.64

$$PE(\%) = \frac{Average - Best}{Best} \times 100\% \quad (12)$$

For Eil51, Eil76, KroA100, and Pr264, SOS-ACO finds the BKS. For the other instances, Best is very close to BKS although SOS-ACO did not find BKS in ten experiments. For all instances, Error(%) did not exceed 3%. It indicates that SOS-ACO can find the solution of high quality in most experiments. In addition, PE(%) is within 1% for all instances. It says that the ten solutions have a small difference. The results illustrate that SOS-ACO has the strong robustness for TSP. The best solutions of several TSP instances computed by SOS-ACO are shown in Fig. 2. One can evaluate the quality of these solutions in view of the pictures.

6.3. Comparison with some other algorithms

Under the same preconditions, we did the experiments for the same TSP instances with SOS-ACO, ACO, and ACO with the local optimization strategy (ACO-LO) and the experimental results are compared. In addition, some hybrid algorithms in the previous literatures, such as PACO-3Opt and HGA, PSO-ACO-3Opt and ACSFA, will be compared with SOS-ACO, respectively.

6.3.1. Comparisons between SOS-ACO and ACO-LO, ACO

We have to give the values of α and β when ACO-LO and ACO are used to resolve TSP. Through experiments, Dorigo et al. [42] found that ACO achieves the better results as $\alpha = 1$ and $\beta = 5$. Due to this study, the values of α and β are taken as 1 and 5, respectively for ACO-LO and ACO in the experiment. The other parameter values are taken from Table 2. For each instance, every algorithm is run ten times. Table 4 shows the experimental results of SOS-ACO, ACO-LO and ACO. The results of SOS-ACO are taken from Table 3.

In view of the results in Table 4, the solutions computed with SOS-ACO for these instances are obviously better than those obtained by ACO-LO and ACO. Due to the function of the local optimization strategy, the solutions based on ACO-LO are better than those of ACO whereas they are worse than those computed with SOS-ACO. It means SOS-ACO is better than ACO-LO and ACO for TSP in most cases. For the bigger scale TSP instances, such as Lin318, Rd400, Pr439 and Rat575, Error(%) computed with ACO-LO and ACO exceeds 7% and 13%, respectively. However, Error(%) computed based on SOS-ACO is less than 2.33%. Compared with ACO-LO and ACO, SOS-ACO can find the solutions of higher quality, especially for large scale instances.

To disclose the solution changes in the optimization process, Fig. 3 shows the evolution of the Best computed with the three algorithms for Eil51, KroA100, KroA200, Lin318, Pr439 and Rat575.

For each instance in Fig. 3, the solution computed by SOS-ACO is superior to that based on ACO-LO and ACO at the same stage. Moreover, there is a big gap between the solutions computed with SOS-ACO, ACO-LO and ACO. It says SOS-ACO has the

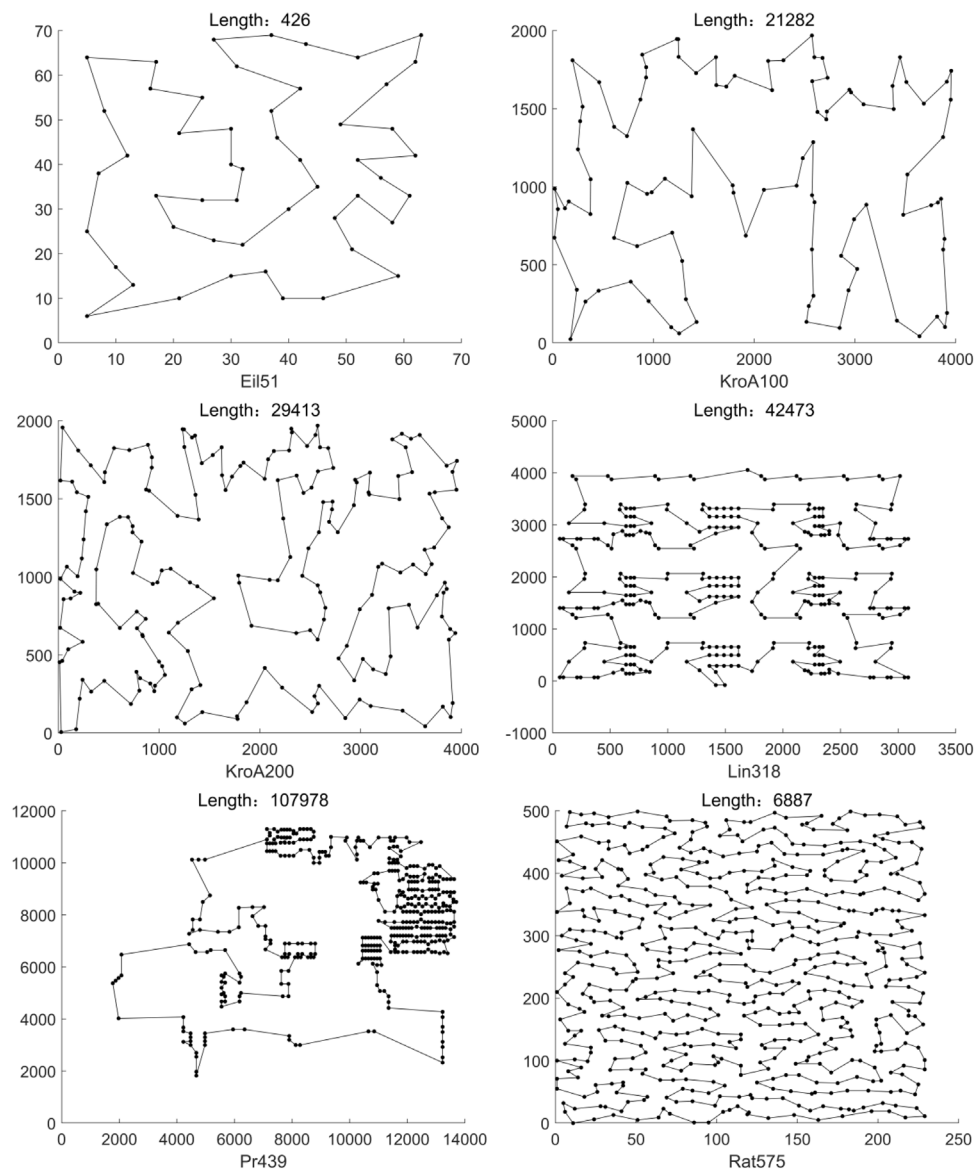


Fig. 2. The best solution computed by SOS-ACO.

Table 4

Comparison between SOS-ACO and ACO-LO, ACO.

TSP	BKS	SOS-ACO			ACO-LO			ACO		
		Best	Average	Error(%)	Best	Average	Error(%)	Best	Average	Error(%)
Eil51	426	426	428.1	0.49	430	436.4	2.44	449	453.2	6.38
Eil76	538	538	541.7	0.69	549	553.2	2.83	570	574.4	6.77
KroA100	21282	21282	21290.1	0.04	21892	22087.8	3.79	23308	23538.6	10.6
Ch150	6528	6558	6571.2	0.66	6637	6689.8	2.48	6808	6852.4	4.97
KroA200	29368	29413	29520.2	0.52	30115	30785.3	4.83	31846	32744.9	11.5
Pr264	49135	49135	49250.7	0.24	52062	52685.4	7.23	54007	54542.8	11.01
Lin318	42029	42473	42762.7	1.75	44849	45544.1	8.36	47659	48512.9	15.43
Rd400	15281	15429	15533	1.65	16430	16499.4	7.97	17568	17806.2	16.53
Pr439	107217	107978	108873.8	1.55	114145	115473	7.7	119980	121390	13.22
Rat575	6773	6887	6930.9	2.33	7364	7505.1	10.81	8053	8155.1	20.41

absolutely better exploration capacity than ACO-LO and ACO. In addition, SOS-ACO converges to the best solution within a smaller number of iterations whereas ACO-LO and ACO consume more cycles to converge. In total, the experimental results show that SOS-ACO can find a better solution than ACO-LO and ACO within the smaller number of iterations. Comparing with ACO and ACO-LO, the experiments show that SOS-ACO has the dual merits of

searching the better solutions and accelerating the convergence rate.

6.3.2. Comparisons between SOS-ACO and some other algorithms

In this section, SOS-ACO is compared with PACO-30pt, HGA, PSO-ACO-30pt and ACSFA in recent literatures. It mentions that PACO-30pt and HGA contain local optimization strategies, and

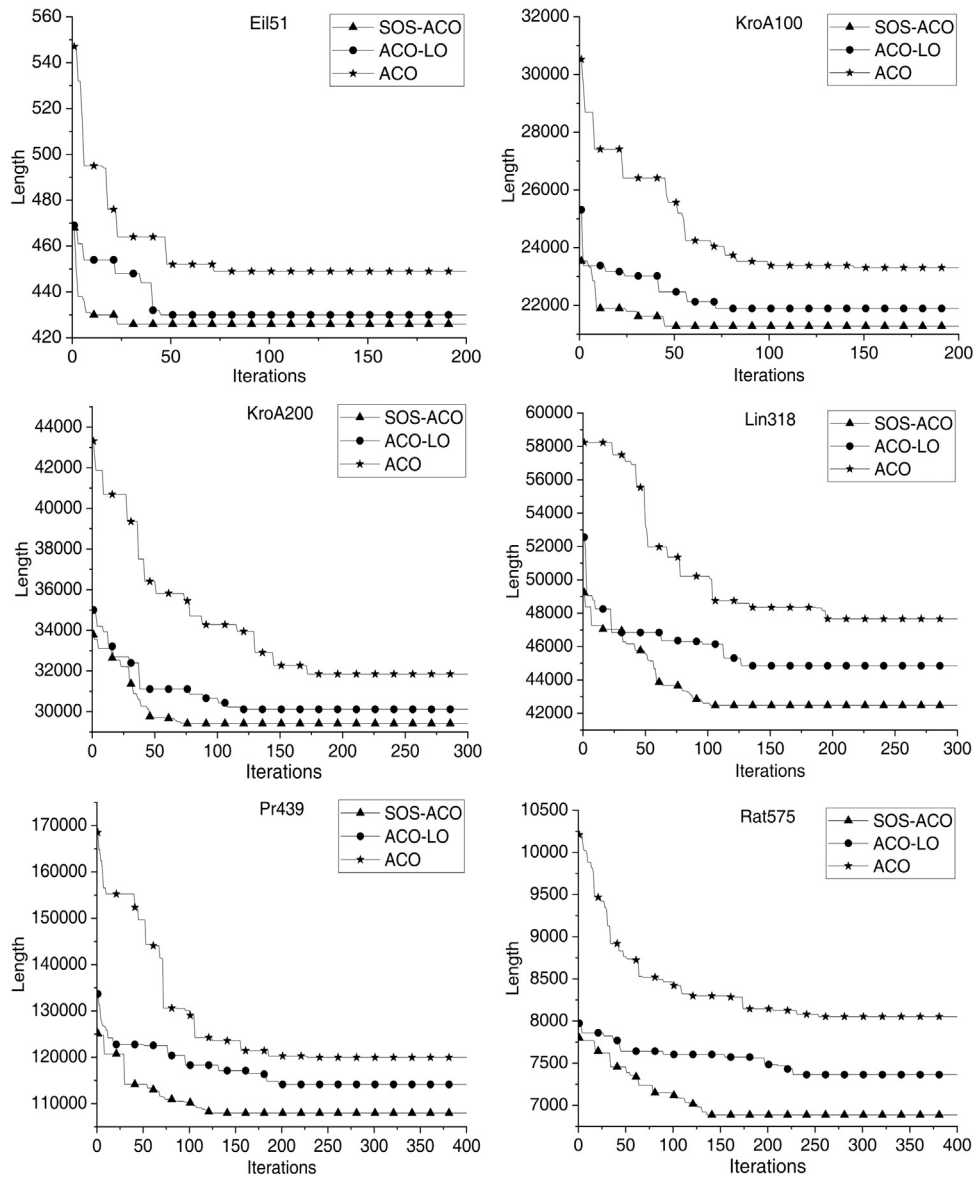


Fig. 3. The evolutionary process of the best solutions.

PSO-ACO-3Opt and ACSFA include ACO parameters optimization. In PACO-3Opt, the solutions are first found by ACO and 3-Opt is used to improve the solutions [43]. HGA is the combination of genetic algorithm and two local optimization strategies [27]. In PSO-ACO-3Opt, PSO is used to optimize the parameters of ACO and 3-Opt is used to improve the solutions [33]. ACSFA is the combination of ACO and firefly algorithm (FA) in which the parameters of ACO are optimized by FA [44]. For the same TSP instances, the experimental results of these algorithms are shown in Table 5. The results of SOS-ACO are taken from Table 3.

For each instance, the best solution *Best* and the minimum value of *Error(%)* computed by the five algorithms are marked in bold in Table 5. The sign ‘-’ indicates that the experimental results relate to the instances are not given in the literatures. Compared with the other four algorithms, SOS-ACO finds eight of the ten best solutions for the ten instances. PACO-3Opt shows the better performance than the other three algorithms and it detects three better solutions. One sees SOS-ACO finds better solutions for most of the TSP instances. Moreover, for the five instances KroA100, Lin318, Rd400, Pr439 and Rat575, the minimum *Error(%)* is obtained according to SOS-ACO. It means that the

average solution found by SOS-ACO in multiple experiments is better than that according to the other four algorithms, especially for the big TSP instances. For example Pr264, Lin318, Rd400, Pr439, and Rat575, SOS-ACO calculates the best solutions better than the other four algorithms. Experimental results show that SOS-ACO is more competitive than the four algorithms for TSP, especially for resolving big scale TSP instances.

6.4. The experiments as the other parameters changing

In SOS-ACO, SOS is used to optimize the key parameters α and β of ACO. The changes of the other parameters may affect the performance of SOS-ACO and the experimental results. In the following, we shall do experiments for several TSP instances as the parameters m , ρ , and Q change. For each instance, SOS-ACO is still run ten times. Based on the experimental results, the performance of SOS-ACO is analyzed according to the changing parameters.

6.4.1. The results according to m

The number of ants is set as 10, 20, and 30, respectively. The other parameter values are taken from Table 2. The experimental

Table 5
Comparisons between SOS-ACO and some other algorithms.

Algorithm	TSP	Eil51	Eil76	KroA100	Ch150	KroA200	Pr264	Lin318	Rd400	Pr439	Rat575
	<i>BKS</i>	426	538	21 282	6528	29 368	49 135	42 029	15 281	107 217	6773
SOS-ACO	<i>Best</i>	426	538	21282	6558	29 413	49135	42473	15429	107978	6887
	<i>Average</i>	428.1	541.7	21290.1	6571.2	29520.2	49250.7	42762.7	15 533	108873.8	6930.9
	<i>Error(%)</i>	0.49	0.69	0.04	0.66	0.52	0.24	1.75	1.65	1.55	2.33
PACO-3Opt [43]	<i>Best</i>	426	538	21282	6570	29533	–	–	15 527	108 482	7003
	<i>Average</i>	426.4	540	21326.8	6601.4	29644.5	–	–	15 601	108 889	7012.4
	<i>Error(%)</i>	0.09	0.37	0.21	1.12	0.94	–	–	2.09	1.56	3.53
HGA [27]	<i>Best</i>	428.9	544.4	21285.4	6530.9	29369.4	49151.2	42624.3	16 050	110171.3	–
	<i>Average</i>	429.2	546.1	21312.5	6557.7	29458.8	49163.3	42877.2	16 144	111 210	–
	<i>Error(%)</i>	0.75	1.51	0.14	0.45	0.31	0.06	2.02	5.65	3.72	–
PSO-ACO-3Opt [33]	<i>Best</i>	426	538	21 301	6538	29468	–	–	–	–	–
	<i>Average</i>	426.5	538.3	21445.1	6564	29646.1	–	–	–	–	–
	<i>Error(%)</i>	0.12	0.06	0.77	0.55	0.95	–	–	–	–	–
ACSFA [44]	<i>Best</i>	428	–	21396	–	–	–	43061	–	–	7097
	<i>Average</i>	432.6	–	21491	–	–	–	43718.6	–	–	7420.3
	<i>Error(%)</i>	1.55	–	0.98	–	–	–	4.02	–	–	9.56

Table 6
The experimental results according to m .

TSP	$m = 10$			$m = 20$			$m = 30$		
	<i>Best</i>	<i>Average</i>	<i>Error(%)</i>	<i>Best</i>	<i>Average</i>	<i>Error(%)</i>	<i>Best</i>	<i>Average</i>	<i>Error(%)</i>
Eil51	426	428.1	0.49	426	428.8	0.66	426	428.2	0.52
KroA100	21 282	21290.1	0.04	21 282	21300.3	0.09	21 282	21309.5	0.13
KroA200	29 413	29520.2	0.52	29 387	29519.8	0.52	29 413	29 502	0.46
Lin318	42 473	42762.7	1.75	42 620	42846.2	1.94	42 640	42865.4	1.99
Pr439	107 978	108873.8	1.55	107 902	108423.6	1.13	107 962	108 663	1.35

Table 7
The experimental results of KroA100 according to $m = 20, 30$.

No.	$m = 20$			$m = 30$		
	<i>Length</i>	α	β	<i>Length</i>	α	β
1	21 282	4.46	3.84	21 282	5.00	3.33
2	21 383	3.85	1.87	21 282	4.36	1.18
3	21 282	3.97	2.49	21 318	3.15	1.54
4	21 282	4.77	2.77	21 282	3.65	5.00
5	21 305	4.24	1.98	21 282	2.02	1.00
6	21 282	4.81	2.72	21 443	4.82	1.52
7	21 318	3.03	3.40	21 292	4.95	1.00
8	21 305	3.86	3.13	21 350	4.21	2.07
9	21 282	4.92	1.53	21 282	2.99	1.08
10	21 282	5.00	3.02	21 282	2.13	3.96

results of SOS-ACO are shown in Table 6. It mentions that the results corresponding to $m = 10$ are taken from Table 3.

When m changes from 10, 20 to 30, *Best*, *Average*, and *Error(%)* computed by SOS-ACO have small difference in ten experiments. *Best* remains nearly unchanged for each TSP instance. It says the solution does not fluctuate much according to the number of ants. However, a big number of ants will increase the computation time. Therefore, it is feasible to set a small number of ants for SOS-ACO.

It mentions that α and β are changing in the optimization process for computing the best solution. Firstly, we show the optimal values of α and β for KroA100 in the ten experiments according to $m = 20$ and 30, respectively. The experimental results are shown in Table 7.

It can be seen that SOS computes different optimal solutions of α and β in various experiment. Although α and β converge at different values, the path length found by SOS-ACO are equal in most cases. For distinctive solutions, their difference is very small. Thus, SOS is important to guarantee the stability of SOS-ACO by adjusting the values of α and β .

Secondly, we show the changes of α and β according to iteration in two experiments. The evolutions of α and β for the 1st experiment when $m = 20$ and 30, respectively in Table 7

are given in Fig. 4. In the early stage, SOS keep searching the better and better parameter values for computing the higher and higher quality of solutions. One sees the parameter values change frequently in this stage. It indicates SOS-ACO does not converge at the optimal solution. In the later stage, SOS converges at the fixed parameter values, which also means that SOS-ACO finally converges at the best solution.

6.4.2. The results according to ρ and Q

First, ρ is set as 0.1, 0.3, 0.5 and 0.7, respectively and the other parameter values are taken from Table 2. SOS-ACO is run ten times and the experimental results are shown in Table 8. *Best* and *Error(%)* are given and one can derive *Average* according to Eq. (11). Secondly, Q is set as 1000, 3000 and 5000, respectively and the other parameters are set as those in Table 2. The experimental results are shown in Table 9. In Tables 8 and 9, the results corresponding to $\rho = 0.1$ and $Q = 1000$ are taken from Table 3.

In view of the results in Table 8, *Best* and *Error(%)* have small difference when ρ changes from 0.1 to 0.3 and 0.5. The performance of SOS-ACO stays unchanged for small and moderate ρ values. As $\rho = 0.7$, the results are slightly worse than those according to $\rho = 0.1, 0.3$ and 0.5. It indicates an excessively big value of ρ will cause SOS-ACO to compute an inferior solution. When ρ is too big, the pheromone on the path will evaporate quickly. In this case, the ants will heavily rely on the heuristic information between cities and they will repeat the shortest paths which are previously traveled. Since no new paths are explored, the algorithm will fall into local minima soon. In addition, even though $\rho = 0.7$, we see that *Error(%)* does not exceed 3% for these instances. That is to say, the average solution computed by SOS-ACO is still close to *BKS*. Thus, the performance of SOS-ACO will not decrease much and it will compute the satisfactory solutions unless ρ is too big.

In the end, the experimental results according to Q are given and analyzed. In Table 9, SOS-ACO calculates the solutions of high quality according to different Q . *Best*, *Average* and *Error(%)* are nearly equal for different values of Q . It means that the performance of SOS-ACO is little affected by the change of Q .

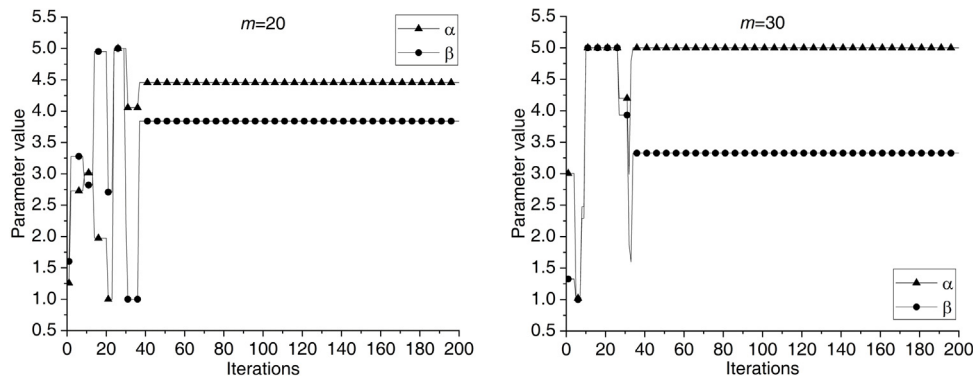
Fig. 4. The evolution of α and β according to m .

Table 8

The experimental results according to ρ .

TSP	$\rho = 0.1$		$\rho = 0.3$		$\rho = 0.5$		$\rho = 0.7$	
	Best	Error(%)	Best	Error(%)	Best	Error(%)	Best	Error(%)
Eil51	426	0.49	426	0.26	426	0.47	426	0.66
KroA100	21282	0.04	21282	0.16	21282	0.07	21282	0.39
KroA200	29413	0.52	29413	0.50	29448	0.53	29496	0.69
Lin318	42473	1.75	42613	1.91	42632	1.94	42780	2.36
Pr439	107978	1.55	107944	1.28	107962	1.33	108232	1.76

Table 9

The experimental results according to Q .

TSP	$Q = 1000$			$Q = 3000$			$Q = 5000$		
	Best	Average	Error(%)	Best	Average	Error(%)	Best	Average	Error(%)
Eil51	426	428.1	0.49	426	427.9	0.45	426	427.8	0.42
KroA100	21282	21290.1	0.04	21282	21295.4	0.06	21282	21301.2	0.09
KroA200	29413	29520.2	0.52	29455	29503	0.46	29496	29521.6	0.52
Lin318	42473	42762.7	1.75	42748	42896.6	2.06	42675	42825.2	1.89
Pr439	107978	108873.8	1.55	107902	108454.2	1.15	108060	108537.4	1.23

Table 10

The experimental results of KroA100 according to ρ .

No.	$\rho = 0.3$			$\rho = 0.5$			$\rho = 0.7$		
	Length	α	β	Length	α	β	Length	α	β
1	21440	5.00	5.00	21282	1.76	2.28	21443	3.30	2.85
2	21282	3.85	1.15	21282	3.40	1.00	21443	4.45	1.09
3	21373	3.23	3.33	21320	5.00	1.94	21305	1.13	4.78
4	21292	1.51	1.47	21292	3.85	2.10	21282	1.09	1.67
5	21282	3.03	1.00	21346	2.21	3.08	21443	1.27	1.05
6	21282	2.06	2.88	21282	1.95	1.00	21443	3.76	1.00
7	21282	3.45	3.23	21320	1.78	2.09	21282	2.32	1.67
8	21282	4.32	1.49	21282	3.48	3.34	21282	2.80	2.42
9	21282	1.53	1.00	21282	4.16	2.06	21282	1.31	1.00
10	21373	3.74	1.31	21282	2.70	3.28	21453	1.82	1.21

Table 11

The experimental results of KroA100 according to Q .

No.	$Q = 3000$			$Q = 5000$		
	Length	α	β	Length	α	β
1	21282	3.69	2.15	21373	1.68	1.79
2	21292	1.68	2.96	21282	3.04	1.90
3	21282	3.04	2.69	21282	2.89	3.40
4	21373	2.14	1.68	21282	3.01	2.02
5	21282	1.78	1.69	21282	2.13	2.66
6	21282	2.13	1.04	21282	4.08	1.79
7	21305	1.24	1.12	21282	3.95	3.90
8	21292	2.98	1.43	21373	1.87	1.23
9	21282	3.27	1.29	21282	2.94	1.00
10	21282	2.79	1.09	21292	1.80	1.00

Taking KroA100 as an example, Tables 10 and 11 list the path length and the optimal values of α and β obtained by SOS-ACO in each experiment when different values of ρ and Q are used.

In view of the results in Tables 10 and 11, SOS converges at different values in the experiments and the path length calculated with SOS-ACO have slightly difference. Fig. 5 shows the evolutions of α and β in the 2nd experiment when ρ is 0.3, the 1st experiment when ρ is 0.5, and the 4th experiment when ρ is 0.7. Fig. 6 shows the evolutions of α and β in the 1st experiment when $Q = 3000$ and the 2nd experiment when $Q = 5000$.

Similar to Fig. 4, in Figs. 5 and 6, α and β are constantly changing in the early stage until SOS-ACO finally converges at the best solution. As the other parameters change, the optimal values of α and β will be computed accordingly for finding the best solution. In the optimization process, SOS adaptively adjusts the

parameter values of α and β according to the solutions searched by the ants and the values of the other parameters, and then lead ant colony to detect the more promising solutions. Thus, the performance of ACO is improved and the negative influence of the other unsuitable parameter values is greatly decreased. The experimental results show that SOS-ACO has good adaptive ability to various values of the other parameters.

7. Conclusions and future research

A hybrid algorithm SOS-ACO is provided for TSP where SOS is used to optimize the two key parameters of ACO which play an important role to let ants find the satisfactory solutions. A local optimization strategy is adopted to accelerate the convergence rate and improve the solution quality. Compared with ACO and

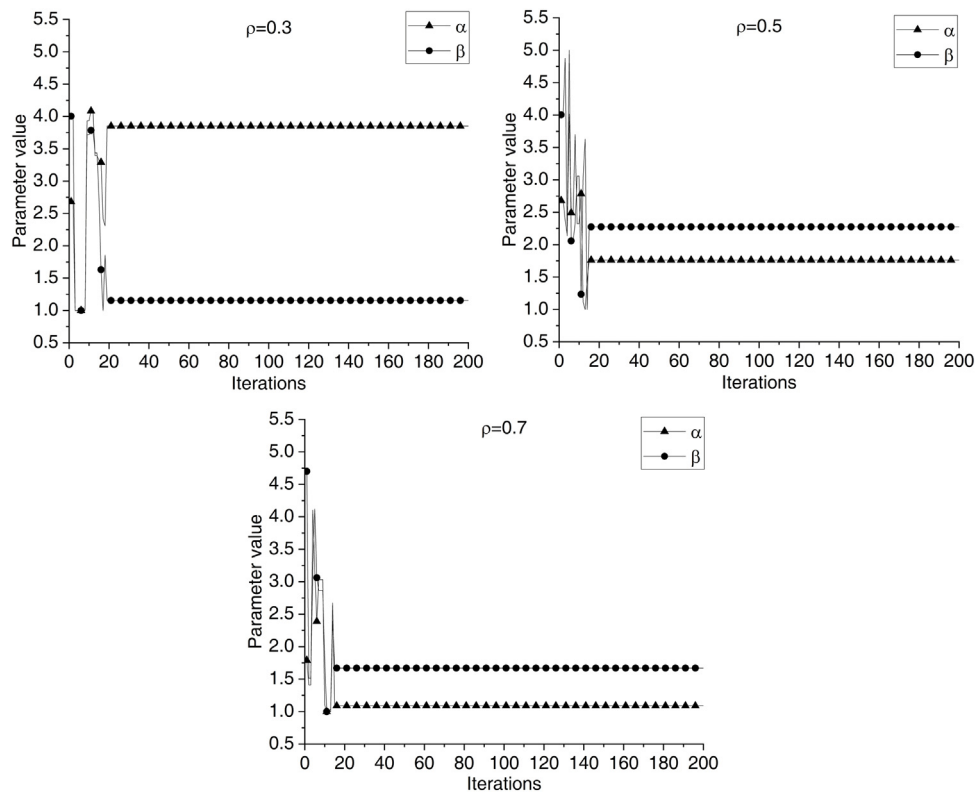


Fig. 5. The evolution of α and β according to ρ .

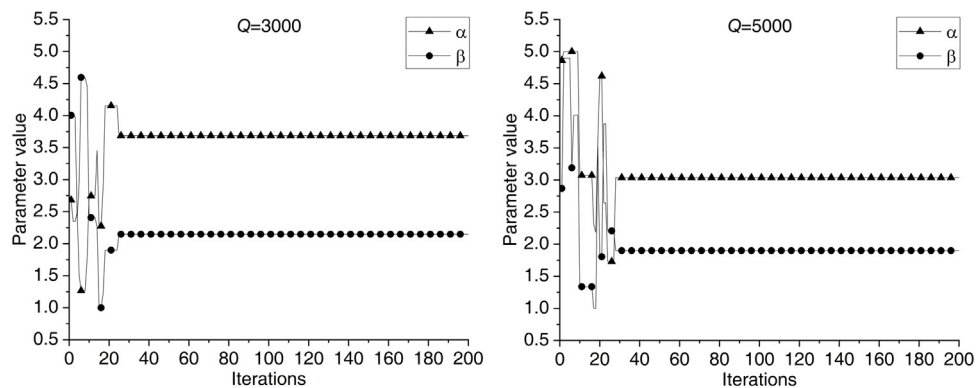


Fig. 6. The evolution of α and β according to Q .

ACO-LO, SOS-ACO finds better solutions within a smaller number of iterations. For some big scale TSP instances, such as Lin318, Rd400, Pr439 and Rat575, *Error*(%) computed based on SOS-ACO is less than 2.33% whereas that obtained by ACO-LO and ACO is over 7% and 13%, respectively. Compared with several other algorithms, SOS-ACO shows stronger optimization capacity and robustness for TSP. Moreover, SOS-ACO illustrates the good adaptive ability for finding the competitive solutions as the other parameters change.

In the future, some other efficient local search strategies, such as *k*-Opt, will be incorporated for improving SOS-ACO. In addition, SOS-ACO will be applied to some related problems of various parallel machines and optoelectronic architectures, such as OTIS-Hypercube, OTIS-Mesh, OTIS hyper hexa-cell, and OTIS mesh of trees [45,46].

CRedit authorship contribution statement

Yong Wang: Conceptualization, Methodology, Software, Writing - original draft, Supervision. **Zunpu Han:** Software, Validation, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors acknowledge the funds supported by the Fundamental Research Funds for the Central Universities, China (Nos. 2018ZD09 and 2018MS039).

References

- [1] G. Gutin, A.P. Punnen, *The Traveling Salesman Problem and its Variations*, Springer, Boston, 2007.
- [2] Y. Wang, J.B. Remmel, A binomial distribution model for the traveling salesman problem based on frequency quadrilaterals, *J. Graph. Algorithms Appl.* 20 (2) (2016) 411–434.
- [3] Muren, J.J. Wu, L. Zhou, Z.P. Du, Y. Lv, Mixed steepest descent algorithm for the traveling salesman problem and application in air logistics, *Transp. Res. E* 126 (2019) 87–102.
- [4] H. Savuran, M. Karakaya, Efficient route planning for an unmanned air vehicle deployed on a moving carrier, *Soft. Comput.* 20 (7) (2016) 2905–2920.
- [5] P. Baniassadi, M. Foumani, K. Smith-Miles, V. Ejov, A transformation technique for the clustered generalized traveling salesman problem with applications to logistics, *European J. Oper. Res.* 285 (2) (2020) 444–457.
- [6] R.M. Karp, On the computational complexity of combinatorial problems, *Networks* 5 (3) (1975) 331–333.
- [7] S. Climer, W.X. Zhang, Cut-and-solve: An iterative search strategy for combinatorial optimization problems, *Artificial Intelligence* 170 (8–9) (2006) 714–738.
- [8] M. Held, R.M. Karp, A dynamic programming approach to sequencing problems, *J. Soc. Ind. Appl. Math.* 10 (1) (1962) 196–210.
- [9] J.J. Grefenstette, R. Gopal, B. Rosmaita, D. Van Gucht, Genetic algorithms for the traveling salesman problem, in: *Proceedings of the First International Conference on Genetic Algorithms and their Applications*, 1985, pp. 160–168.
- [10] Y.W. Zhong, J. Lin, L.J. Wang, H. Zhang, Discrete comprehensive learning particle swarm optimization algorithm with Metropolis acceptance criterion for traveling salesman problem, *Swarm. Evol. Comput.* 42 (2018) 77–88.
- [11] X.T. Geng, Z.H. Chen, W. Yang, D.Q. Shi, K. Zhao, Solving the traveling salesman problem based on an adaptive simulated annealing algorithm with greedy search, *Appl. Soft. Comput.* 11 (4) (2011) 3680–3689.
- [12] M. Dorigo, L.M. Gambardella, Ant colonies for the traveling salesman problem, *Biosystems* 43 (2) (1997) 73–81.
- [13] W. Elloumi, H. El Abed, A. Abraham, A.M. Alimi, A comparative study of the improvement of performance using a PSO modified by ACO applied to TSP, *Appl. Soft. Comput.* 25 (2014) 234–241.
- [14] I. Khan, M.K. Maiti, A swap sequence based artificial bee colony algorithm for traveling salesman problem, *Swarm. Evol. Comput.* 44 (2019) 428–438.
- [15] M.A.H. Akhand, S.I. Ayon, S.A. Shahriyar, N. Siddique, H. Adeli, Discrete spider monkey optimization for travelling salesman problem, *Appl. Soft. Comput.* 86 (2020) 105887.
- [16] E. Osaba, J.D. Ser, A. Sadollah, M.N. Bilbao, D. Camacho, A discrete water cycle algorithm for solving the symmetric and asymmetric traveling salesman problem, *Appl. Soft. Comput.* 71 (2018) 277–290.
- [17] B.A. Mahafzah, Performance evaluation of parallel multithreaded A* heuristic search algorithm, *J. Inf. Sci.* 40 (3) (2014) 363–375.
- [18] B.A. Mahafzah, Parallel multithreaded IDA* heuristic search: Algorithm design and performance evaluation, *Int. J. Parallel Emerg.* 26 (1) (2011) 61–82.
- [19] H. Chen, W. Li, X. Yang, A whale optimization algorithm with chaos mechanism based on quasi-opposition for global optimization problems, *Expert Syst. Appl.* 158 (2020) 113612.
- [20] R. Masadeh, A. Sharieh, B.A. Mahafzah, Humpback whale optimization algorithm based on vocal behavior for task scheduling in cloud computing, *Int. J. Adv. Sci. Tech.* 13 (3) (2019) 121–140.
- [21] S. Gupta, K. Deep, A memory-based grey wolf optimizer for global optimization tasks, *Appl. Soft. Comput.* 93 (2020) 106367.
- [22] C. Liu, Y.K. Du, A membrane algorithm based on chemical reaction optimization for many-objective optimization problems, *Knowl.-Based Syst.* 165 (2019) 306–320.
- [23] R.Y. Dong, S.S. Wang, G.Y. Wang, X.Y. Wang, Hybrid optimization algorithm based on wolf pack search and local search for solving traveling salesman problem, *J. Shanghai Jiao Tong Univ. (Sci.)* 24 (1) (2019) 41–47.
- [24] G.A. Croes, A method for solving traveling salesman problems, *Oper. Res.* 6 (6) (1958) 791–812.
- [25] S. Hougardy, F. Zaiser, X.H. Zhong, The approximation ratio of the 2-Opt Heuristic for the metric Traveling Salesman Problem, *Oper. Res. Lett.* 48 (2020) 401–404.
- [26] F. Dahan, K. El Hindi, H. Mathkour, H. AlSalman, Dynamic flying ant colony optimization (DFACO) for solving the traveling salesman problem, *Sensors* 19 (8) (2019) 1–28.
- [27] Y. Wang, The hybrid genetic algorithm with two local optimization strategies for traveling salesman problem, *Comput. Ind. Eng.* 70 (4) (2014) 124–133.
- [28] M. Mavrovouniotis, F.M. Müller, S. Yang, Ant colony optimization with local search for dynamic traveling salesman problems, *IEEE. Trans. Cybern.* 47 (7) (2017) 1743–1756.
- [29] M.B. Ghalia, Particle swarm optimization with an improved exploration-exploitation balance, in: *2008 51st Midwest Symposium on Circuits and Systems*, 2008, pp. 759–762.
- [30] X.S. Yang, S. Deb, M. Loomes, M. Karamanoglu, A framework for self-tuning optimization algorithm, *Neural Comput. Appl.* 23 (7–8) (2013) 2051–2057.
- [31] M. Peker, B. Sen, P.Y. Kumru, An efficient solving of the traveling salesman problem: the ant colony system having parameters optimized by the Taguchi method, *Turk. J. Electr. Eng. Comput.* 21 (2013) 2015–2036.
- [32] Y. Gan, L.W. Lan, S. Li, Study on parameters configuration for ant colony optimization, *Adv. Mater. Res.* 279 (2011) 371–376.
- [33] M. Mahi, Ö.K. Baykan, H. Kodaz, A new hybrid method based on particle swarm optimization, ant colony optimization and 3-opt algorithms for traveling salesman problem, *Appl. Soft. Comput.* 30 (2015) 484–490.
- [34] H.M. Botee, E. Bonabeau, Evolving ant colony optimization, *Adv. Complex. Syst.* 1 (1998) 149–159.
- [35] A.H. Erol, M. Er, S. Bulkan, Optimizing the ant colony optimization algorithm using neural network for the traveling salesman problem, in: *Proceedings of the 2012 International Conference on Industrial Engineering and Operations Management*, IEOM, 2012, pp. 1695–1701.
- [36] M.Y. Cheng, D. Prayogo, Symbiotic Organisms Search: A new metaheuristic optimization algorithm, *Comput. Struct.* 139 (2014) 98–112.
- [37] E.S. Ezugwu, A.O. Adewumi, Discrete symbiotic organisms search algorithm for traveling salesman problem, *Expert Syst. Appl.* 87 (2017) 70–78.
- [38] Y. Wang, Y.W. Wu, N. Xu, Discrete symbiotic organism search with excellence coefficients and self-escape for traveling salesman problem, *Comput. Ind. Eng.* 131 (2019) 269–281.
- [39] A. Panda, S. Pani, A Symbiotic Organisms Search algorithm with adaptive penalty function to solve multi-objective constrained optimization problems, *Appl. Soft. Comput.* 46 (2016) 344–360.
- [40] H.S. Wu, F.M. Zhang, H. Li, X.L. Liang, Discrete wolf pack algorithm for traveling salesman problem, *Control. Decis.* 30 (2015) 1861–1867.
- [41] G. Reinelt, TSPLIB- a traveling salesman problem library, *ORSA J. Comput.* 3 (1991) 376–384.
- [42] M. Dorigo, G.D. Caro, L.M. Gambardella, Ant algorithms for discrete optimization, *Artif. Life* 5 (2) (1999) 137–172.
- [43] S. Gülcü, M. Mahi, Ö.K. Baykan, H. Kodaz, A parallel cooperative hybrid method based on ant colony optimization and 3-opt algorithm for solving traveling salesman problem, *Soft. Comput.* 22 (5) (2016) 1669–1685.
- [44] M.K.A. Ariyaratne, T.G.I. Fernando, S. Weerakoon, A self-tuning firefly algorithm to tune the parameters of ant colony system, *Int. J. Swarm. Intell.* 3 (4) (2018) 309–331.
- [45] A. Al-Adwan, B.A. Mahafzah, A. Sharieh, Solving traveling salesman problem using parallel repetitive nearest neighbor algorithm on OTIS-hypercube and OTIS-Mesh optoelectronic architectures, *J. Supercomput.* 74 (2018) 1–36.
- [46] A. Al-Adwan, A. Sharieh, B.A. Mahafzah, Parallel heuristic local search algorithm on OTIS hyper hexa-cell and OTIS mesh of trees optoelectronic architectures, *Appl. Intell.* 49 (2019) 661–688.